

Facet: Shared Core Architecture — Rust as Cross- Platform Foundation for Helix VPN

Facet: Shared Core Architecture — Rust as Cross- Platform Foundation for Helix VPN

Revision: 1 Last modified: 2026-07-04T14:00:00Z

Editorial note (added during the 2026-07-04 MVP2 gap-analysis/hardening pass): raw research brief preserved as historical input, not a living spec. Note the code-reuse percentages here (60-70% mobile, 40-50% web) are early estimates; the final, reconciled figures — 72% Android/iOS, 70% HarmonyOS, 45% Web — are specified in `../MVP2_ARCHITECTURE.md §4` and `../MVP2_SHARED_CORE.md §1.3`, which are authoritative where they differ from this brief.

Executive Summary

This research evaluates Rust as a shared core library foundation for maximum code reuse across all Helix VPN target platforms: macOS, Windows, Linux, Android, iOS, HarmonyOS, Aurora OS, and Web. The analysis covers FFI patterns, binding generators (UniFFI, flutter_rust_bridge, wasm-bindgen), build systems, real-world case studies from Mullvad VPN, Signal, Cloudflare BoringTun, and provides a recommended architecture with estimated code reuse percentages per platform.

Key Finding: A well-architected Rust shared core can achieve **75-85% code reuse** across desktop platforms, **60-70%** on mobile (Android/iOS), and **40-50%** on Web (WASM), with HarmonyOS/ Aurora OS achieving **60-70%** through Linux-compatible targets.

Table of Contents

- 1. [Rust Cross-Compilation Target Matrix](#)
- 2. [UniFFI: Mozilla's Multi-Language Binding Generator](#)
- 3. [FFI Patterns and Best Practices](#)
- 4. [Android Rust Integration](#)
- 5. [iOS Rust Integration](#)
- 6. [Desktop Rust Integration](#)
- 7. [WASM Target for Browser/Extensions](#)
- 8. [Real-World Case Studies](#)
- 9. [VPN-Specific Rust Crates](#)
- 10. [Build System Integration](#)
- 11. [Binary Size Optimization](#)
- 12. [HarmonyOS and Aurora OS Support](#)
- 13. [Recommended Architecture for Helix VPN](#)
- 14. [Code Reuse Estimates](#)

1. Rust Cross-Compilation Target Matrix

1.1 Supported Target Triples for All Platforms

Rust's LLVM-based compiler backend provides first-class cross-compilation support. The following target triples are relevant for Helix VPN [^342 ^]:

Platform	Target Triple	Tier	Status
macOS (Intel)	x86_64-apple-darwin	Tier 2	Production-ready
macOS (Apple Silicon)	aarch64-apple-darwin	Tier 2	Production-ready
iOS (Device)	aarch64-apple-ios	Tier 2	Production-ready
iOS (Simulator - Apple Silicon)	aarch64-apple-ios-sim	Tier 2	Production-ready
iOS (Simulator - Intel)	x86_64-apple-ios	Tier 2	Production-ready
Windows (x64)	x86_64-pc-windows-msvc	Tier 1	Full support

Platform	Target Triple	Tier	Status
Windows (ARM64)	aarch64-pc-windows-msvc	Tier 2	Supported
Linux (x64)	x86_64-unknown-linux-gnu	Tier 1	Full support
Linux (ARM64)	aarch64-unknown-linux-gnu	Tier 1	Full support
Android (ARM64)	aarch64-linux-android	Tier 2	Production-ready
Android (ARMEabi)	armv7-linux-androideabi	Tier 2	Production-ready
Android (x86_64)	x86_64-linux-android	Tier 2	Production-ready
Web/Browser	wasm32-unknown-unknown	Tier 2	Production-ready
HarmonyOS	aarch64-unknown-linux-ohos	Tier 2	Supported with SDK setup

1.2 Cross-Compilation Tools

cross-rs (formerly cargo-cross)

The community-maintained wrapper around cargo that makes cross-compilation painless using Docker containers [^367^]:

```
# Install
```

```
cargo install cross --git https://github.com/cross-rs/cross
```

```
# Build for any target
```

```
cross build --release --target aarch64-linux-android
```

```
cross build --release --target aarch64-apple-ios
```

```
cross build --release --target x86_64-pc-windows-msvc
```

Key advantage: Provides complete cross-compilation environments with correct toolchains and system libraries, eliminating manual linker configuration [^367^].

rustup Target Management

```
# Add all required targets for mobile development
```

```
rustup target add aarch64-apple-ios aarch64-apple-ios-sim x86_64-apple-ios
```

```
rustup target add aarch64-linux-android armv7-linux-androideabi x86_64-linux-android
```

```
rustup target add x86_64-pc-windows-msvc aarch64-pc-windows-msvc
```

```
rustup target add wasm32-unknown-unknown
```

2. UniFFI: Mozilla's Multi-Language Binding Generator

2.1 Overview

UniFFI is a toolkit for building cross-platform software components in Rust, developed by Mozilla and used extensively in Firefox mobile and desktop browsers [^338^]. It automatically generates foreign-language bindings, eliminating the need to write FFI plumbing by hand.

"Instead, uniffi does most of the work for us by generating the plumbing necessary to transport data across the FFI, including the specific language bindings, making it a little easier to write things once and a lot easier to maintain multiple supported languages." [^336^]

2.2 Supported Languages

Language	Support Level	Notes
Kotlin	Full (1st party)	Android primary target
Swift	Full (1st party)	iOS/macOS primary target
Python	Full (1st party)	Desktop scripting
Ruby	Legacy/partial	Community maintained
C#	3rd party	Community bindings available
Go	3rd party	Community bindings available
Dart	3rd party	Via flutter_rust_bridge
JavaScript	3rd party	WASM + React Native TurboModules

2.3 Interface Definition

UniFFI supports two interface definition approaches [^340^]:

UDL (UniFFI Definition Language) - WebIDL-based:

```
namespace helixvpn {
    string connect_to_server(string server_address);
    void disconnect();
    TunnelState get_tunnel_state();
};
```

```
interface TunnelConfig {
    constructor(string server_address, u16 port);
    string get_server_address();
};

enum TunnelState {
    "Disconnected",
    "Connecting",
    "Connected",
    "Error"
};
```

Proc-Macro Approach (modern, preferred):

```
#[uniffi::export]
pub fn connect_to_server(server_address: String) -> String {
    // implementation
}

#[derive(uniffi::Object)]
pub struct TunnelConfig {
    server_address: String,
    port: u16,
}

#[uniffi::export]
impl TunnelConfig {
    #[uniffi::constructor]
    pub fn new(server_address: String, port: u16) -> Self {
        Self { server_address, port }
    }
}
```

2.4 Maturity and Production Usage

- **Mozilla Firefox:** Used in Glean SDK (telemetry), Nimbus SDK (experimentation), and application-services [^336^] [^338^]
- **Production readiness:** "We consider it ready for production use" [^338^]
- **Version status:** Pre-1.0, but actively maintained with breaking changes minimized for simple consumers
- **Community ecosystem:** Third-party bindings for C#, Go, Dart, Java, Node.js [^338^]

2.5 Limitations

- Async support is evolving: "The futures/async support is quite immature" [^334^]
- Advanced features may break between upgrades [^338^]

- Some advanced FFI patterns (e.g., custom tokio runtime management) may require manual FFI [^357^]

2.6 Alternative: BoltFFI

A newer high-performance alternative claiming significant speed improvements [^341^]:

- `echo_i32`: <1 ns vs 1,416 ns (UniFFI) — **>1000x faster**
 - Supports Swift, Kotlin, TypeScript (WASM) with Python planned
 - Handles XCFramework generation for Apple platforms natively
-

3. FFI Patterns and Best Practices

3.1 Core Principles

1. **Minimize unsafe code**: Isolate unsafe to thin FFI boundary layers
2. **C ABI compatibility**: Use `#[no_mangle] pub extern "C"` for all exported functions
3. **Memory safety**: Define clear ownership semantics across the boundary
4. **Error handling**: Convert Rust Result to language-appropriate error types

3.2 Async Rust Across FFI

The `async-ffi` crate provides FFI-compatible Futures [^330^]:

```
use async_ffi::{FfiFuture, FutureExt};

#[no_mangle]
pub extern "C" fn work(arg: u32) -> FfiFuture<u32> {
    async move {
        let ret = do_some_io(arg).await;
        ret
    }.into_ffi()
}
```

Key considerations [^329^] [^333^]:

- Use a static/shared Tokio Runtime rather than creating one per call
- Thread-local storage won't work correctly across dynamic library boundaries
- Pass runtime handles explicitly via FFI if needed

Tokio Runtime pattern for FFI [^329^]:

```
use std::sync::LazyLock;
use tokio::runtime::Runtime;

static RUNTIME: LazyLock<Runtime> = LazyLock::new(|| {
    Runtime::new_multi_thread()
        .worker_threads(4)
        .enable_all()
        .build()
        .expect("Failed to create Tokio runtime")
});

#[no_mangle]
pub extern "C" fn vpn_connect(config: *const c_char) -> i32 {
    let config_str = unsafe {
        CString::from_ptr(config).to_string_lossy() };
    match RUNTIME.block_on(async_connect(config_str.as_ref())) {
        Ok(_) => 0,
        Err(e) => e.code(),
    }
}
```

3.3 Error Handling Across Boundaries

Pattern 1: Error codes with thread-local error messages [^331^]:

```
thread_local! { static LAST_ERROR: RefCell<Option<String>> =
    RefCell::new(None); }

#[no_mangle]
pub extern "C" fn get_last_error() -> *mut c_char {
    LAST_ERROR.with(|e| {
        e.borrow().as_ref()
            .map(|s| CString::new(s.as_str()).unwrap().into_raw())
            .unwrap_or(std::ptr::null_mut())
    })
}
```

Pattern 2: Return structs with error info:

```
#[repr(C)]
pub struct FfiResult<T> {
    pub data: T,
    pub error_code: i32,
    pub error_message: *mut c_char,
}
```

Pattern 3: UniFFI-style (recommended): UniFFI automatically converts Rust `Result<T, E>` to exceptions/errors in target languages [^340^].

3.4 Memory Management Best Practices

- **String passing:** Rust returns `CString` → foreign language frees via dedicated function
 - **Object handles:** Use opaque pointer types `(*mut c_void)` with create/destroy functions
 - **Avoid panics across FFI:** Use `catch_unwind` at the boundary (undefined behavior otherwise)
 - **Zero-copy where possible:** Use `&[u8]` slices for packet data transfer
-

4. Android Rust Integration

4.1 Toolchain: cargo-ndk

`cargo-ndk` is the standard tool for building Rust for Android [^348^]:

Install

```
cargo install cargo-ndk
```

Add Android targets

```
rustup target add aarch64-linux-android armv7-linux-androideabi  
x86_64-linux-android
```

Build for all architectures

```
cargo ndk --target aarch64-linux-android --android-platform 33 --  
build --release  
cargo ndk --target armv7-linux-androideabi --android-platform 33  
-- build --release  
cargo ndk --target x86_64-linux-android --android-platform 33 --  
build --release
```

Key features [^348^]:

- Automatically finds and sets NDK linker and AR tools
- Handles sysroot configuration
- Supports API level specification

4.2 JNI Bindings: jni-rs

The `jni` crate (version 0.21+) provides Rust JNI bindings [^344^]:

```
use jni::JNIEnv;  
use jni::objects::JClass;
```

```

use jni::signature::JavaType;

#[no_mangle]
pub extern "C" fn Java_com_helixvpn_core_RustBridge_connect(
    mut env: JNIEnv,
    _class: JClass,
    server_address: JString,
) -> jint {
    let addr: String = env.get_string(&server_address)
        .expect("Invalid string")
        .into();

    match vpn_connect(&addr) {
        Ok(_) => 0,
        Err(e) => {
            env.throw_new("com/helixvpn/core/VpnException",
                e.to_string())
                .unwrap();
            -1
        }
    }
}

```

4.3 Gradle Plugin Integration

The Cargo NDK Gradle Plugin automates Rust builds within Android projects [^346^]:

```

// root build.gradle
buildscript {
    dependencies {
        classpath
            "gradle.plugin.com.github.willir.rust:plugin:0.3.4"
    }
}

// app/build.gradle
apply plugin: "com.github.willir.rust.cargo-ndk-android"

cargoNdk {
    targets = ["arm64", "arm", "x86_64"]
    module = "../rust-core"
    apiLevel = 24
    buildTypes {
        release { buildType = "release" }
        debug { buildType = "debug" }
    }
}

```

This plugin [^346^]:

- Runs automatically during `./gradlew assembleDebug / assembleRelease`
- Copies `.so` files to `jniLibs/` directory
- Supports per-build-type configuration
- Allows target filtering via gradle properties (`-Prust-target=arm64`)

4.4 Android NDK Rust Toolchain Support

- NDK r23+ provides improved LLVM toolchain compatibility
- Rust Android targets use the NDK's Clang linker
- Minimum supported API level: 24 (Android 7.0) for full Rust std support [^344^]

4.5 .so Library Packaging

Output structure:

```
android/app/src/main/jniLibs/
├─ arm64-v8a/libhelix_core.so      (from aarch64-linux-android)
├─ armeabi-v7a/libhelix_core.so   (from armv7-linux-
androideabi)
└─ x86_64/libhelix_core.so        (from x86_64-linux-android)
```

5. iOS Rust Integration

5.1 Modern Approach: XCFramework + UniFFI

The modern recommended approach uses `xcodebuild -create-xcframework` with UniFFI-generated bindings, replacing the deprecated `cargo-lipo` tool [^349^]:

1. Add iOS targets

```
rustup target add aarch64-apple-ios aarch64-apple-ios-sim
```

2. Build for device

```
cargo build --release --target=aarch64-apple-ios
```

3. Build for simulator

```
cargo build --release --target=aarch64-apple-ios-sim
```

4. Generate UniFFI bindings

```
cargo run --bin uniffi-bindgen generate \
  --library ./target/aarch64-apple-ios/release/libhelix.dylib \
  --language swift --out-dir ./bindings
```

```
# 5. Rename modulemap (critical step!)
mv bindings/helixFFI.modulemap bindings/module.modulemap

# 6. Create XCFramework
xcodebuild -create-xcframework \
  -library ./target/aarch64-apple-ios-sim/release/libhelix.a
  -headers ./bindings \
  -library ./target/aarch64-apple-ios/release/libhelix.a
  -headers ./bindings \
  -output ios/HelixCore.xcframework
```

5.2 Swift Integration

After importing the XCFramework and generated Swift file [^82^]:

```
import SwiftUI
import helixFFI // The generated UniFFI module

struct ContentView: View {
    @State private var status = "Disconnected"

    func connect() {
        // Call Rust function through UniFFI-generated bindings
        let result = connectToServer(serverAddress: "us-
        east.helix.vpn")
        status = "Connected"
    }
}
```

5.3 Legacy: cargo-lipo (Deprecated)

cargo-lipo was previously used to create universal fat binaries but is now deprecated because arm64 can represent both device and simulator targets [^349^]. Use XCFrameworks instead.

5.4 cbindgen for C Headers

When not using UniFFI, cbindgen generates C headers from Rust code:

```
cargo install cbindgen --output include/helix_core.h
```

Used by Signal's libsignal for manual bridge layers [^362^].

5.5 Key Considerations for iOS

- **XCFramework is required** (not optional) for Apple Silicon Macs since the same architecture (arm64) is used for both devices and simulators [^349^]

- **Module map renaming:** Must rename xxxFFI.modulemap to module.modulemap for Xcode detection [^82^]
 - **libresolv.tbd:** Must be linked on iOS for DNS resolution [^345^]
 - **Bitcode:** Disabled by default in modern Xcode; Rust binaries are not bitcode-compatible
-

6. Desktop Rust Integration

6.1 Tauri Commands

Tauri provides a Rust backend with web frontend, communicating via typed commands [^10^]:

```
// src-tauri/src/main.rs
#[tauri::command]
async fn vpn_connect(state: tauri::State<'_, VpnState>, server:
    String) -> Result<String, String> {
    state.core.connect(&server).await
    .map(|_| "Connected".into())
    .map_err(|e| e.to_string())
}

fn main() {
    tauri::Builder::default()
        .manage(VpnState::default())
        .invoke_handler(tauri::generate_handler![vpn_connect,
            vpn_disconnect])
        .run(tauri::generate_context!())
        .expect("error while running tauri application");
}
```

Frontend (TypeScript):

```
import { invoke } from '@tauri-apps/api/core';

async function connect() {
    const result = await invoke('vpn_connect', { server: 'us-
        east.helix.vpn' });
    console.log(result);
}
```

Tauri advantages [^10^]:

- Tiny bundle sizes (2-10 MB)
- Native memory efficiency
- Full OS API access through Rust
- Desktop is production-stable; mobile support in beta

6.2 Flutter Desktop with Rust

Two approaches for Flutter + Rust on desktop:

Approach A: flutter_rust_bridge (Recommended)

The flutter_rust_bridge crate is a Flutter Favorite and generates high-level bindings [^392^]:

```
# One-liner setup
cargo install flutter_rust_bridge_codegen &&
    flutter_rust_bridge_codegen create my_app
```

Approach B: Direct FFI via dart:ffi [^379^]:

```
// Load the Rust library
final lib = Platform.isAndroid
    ? DynamicLibrary.open('libhelix_core.so')
    : Platform.isIOS
        ? DynamicLibrary.process() // Statically linked
        : DynamicLibrary.open('libhelix_core.dylib'); // macOS

final connect = lib.lookupFunction<ConnectNative,
    ConnectDart>('rust_connect');
```

Desktop library locations:

- **macOS:** HelixVPN.app/Contents/Frameworks/libhelix_core.dylib
- **Windows:** Place .dll next to executable or in PATH
- **Linux:** Standard library path or LD_LIBRARY_PATH

6.3 Direct Dynamic Library Loading

For native desktop apps (non-Flutter, non-Tauri):

```
// Loading Rust core as .so/.dll/.dylib
#[cfg(target_os = "macos")]
const LIB_PATH: &str = "libhelix_core.dylib";
#[cfg(target_os = "linux")]
const LIB_PATH: &str = "libhelix_core.so";
#[cfg(target_os = "windows")]
const LIB_PATH: &str = "helix_core.dll";

let lib = unsafe { libloading::Library::new(LIB_PATH)? };
let connect: Symbol<unsafe extern "C" fn(*const c_char) -> i32> =
    unsafe { lib.get(b"vpn_connect")? };
```

7. WASM Target for Browser/Extensions

7.1 wasm-bindgen + wasm-pack

The standard Rust-to-WASM toolchain [^353^] [^354^]:

```
# Install
rustup target add wasm32-unknown-unknown
cargo install wasm-pack

# Build for browser
wasm-pack build --target web --out-dir pkg/

# Build for bundler (webpack/rollup)
wasm-pack build --target bundler --out-dir pkg/
```

7.2 VPN Core in WebAssembly

For a browser extension or web UI, the Rust VPN core compiles to WASM [^354^]:

```
// src/lib.rs
use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub struct WasmVpnCore {
    inner: VpnCore,
}

#[wasm_bindgen]
impl WasmVpnCore {
    #[wasm_bindgen(constructor)]
    pub fn new() -> Self {
        console_error_panic_hook::set_once();
        Self { inner: VpnCore::new() }
    }

    pub async fn connect(&self, server: String) ->
        Result<JsValue, JsValue> {
        self.inner.connect(&server).await
            .map(|state| JsValue::from_str(&format!("{:?}",
                state)))
            .map_err(|e| JsValue::from_str(&e.to_string()))
    }
}
```

7.3 Web Limitations for VPN

Critical: WASM in browsers **cannot** create raw network sockets or TUN interfaces due to browser sandboxing. For a browser extension VPN:

1. **Use proxy-based approach:** Configure browser proxy settings to route through a VPN gateway
2. **WebRTC for P2P:** Use WebRTC data channels for VPN tunneling
3. **Native messaging:** Communicate with a native host application that runs the actual VPN core
4. **WASM for crypto:** Use Rust crypto (Noise protocol, ChaCha20-Poly1305) in WASM while delegating networking to JavaScript APIs

7.4 Browser Extension Architecture

Browser Extension Architecture:

Popup UI (React/Vue) └ Calls WASM crypto functions
Background Service Worker └ chrome.proxy API or Native Messaging
Native Host App (Rust binary, platform-specific) └ Real TUN interface, WireGuard, etc.

7.5 Performance Considerations

- WASM has ~1.5-2x overhead vs native Rust [^353^]
 - Crypto operations in WASM are still faster than pure JavaScript
 - Use `wasm-opt` (Binaryen) for additional optimization
 - Enable `wee_alloc` or `dlmalloc` as smaller WASM allocators
-

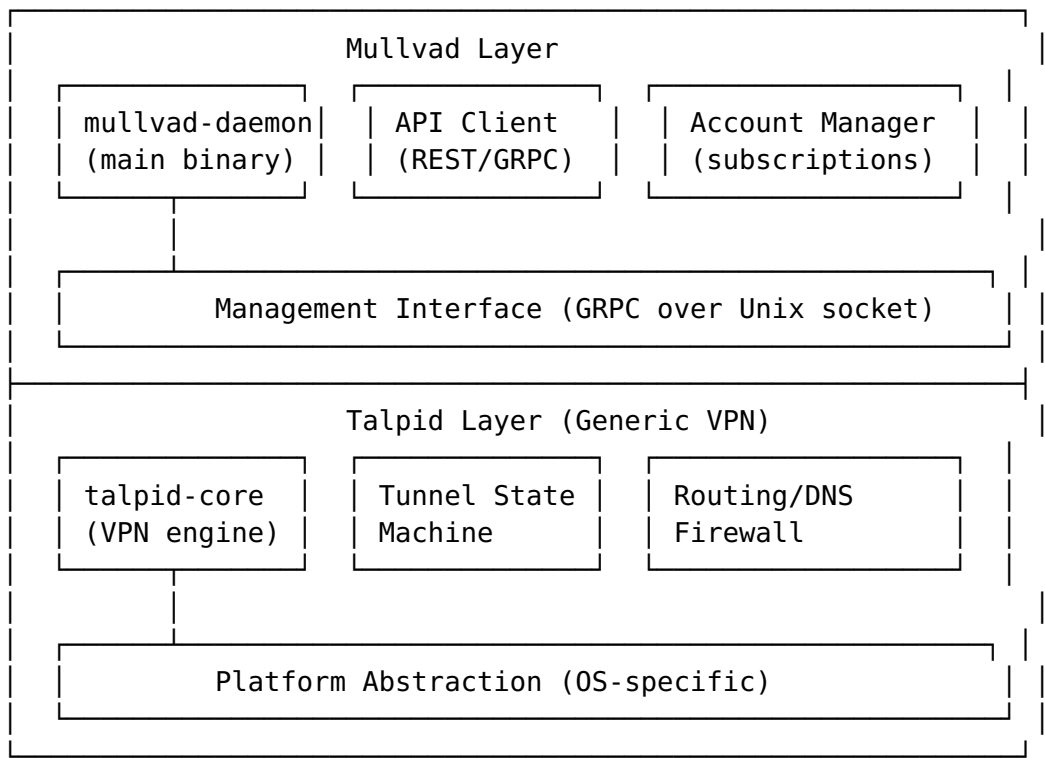
8. Real-World Case Studies

8.1 Mullvad VPN — Rust Core Architecture

Repository: github.com/mullvad/mullvadvpn-app (7.3k+ stars) [^155^]

Architecture Overview

Mullvad's architecture splits into two distinct layers [^23^]:



Key Design Principles

"The talpid crates are supposed to be completely unrelated to Mullvad specific things. A talpid crate is not allowed to know anything about the API through which the daemon fetch Mullvad account details or download VPN server lists." [^23^]

- **talpid-core:** Generic VPN client library, completely Mullvad-agnostic [^155^]
- **mullvad-daemon:** Mullvad-specific daemon binary
- **Frontend communication:** GRPC over Unix socket (desktop), JNI (Android), standalone iOS

Platform Split

Platform	Architecture	Code Reuse
Desktop (macOS/Windows/Linux)	Shared mullvad-daemon Rust binary + Electron GUI	~85%
Android	Same Rust core via JNI + Kotlin UI	~70%
iOS		Separate codebase

Platform	Architecture	Code Reuse
	Standalone Swift implementation in ios/ directory	

GotaTun — WireGuard in Rust

Mullvad forked Cloudflare's BoringTun to create GotaTun [^363^] [^61^]:

"GotaTun integrates additional functionality like DAITA and Multihop compared to Cloudflare's BoringTun code... Previously Mullvad was relying on a Go language implementation of WireGuard. With the WireGuard Go implementation they had encountered crashes while so far 'not a single crash' has been detected with GotaTun."
[^61^]

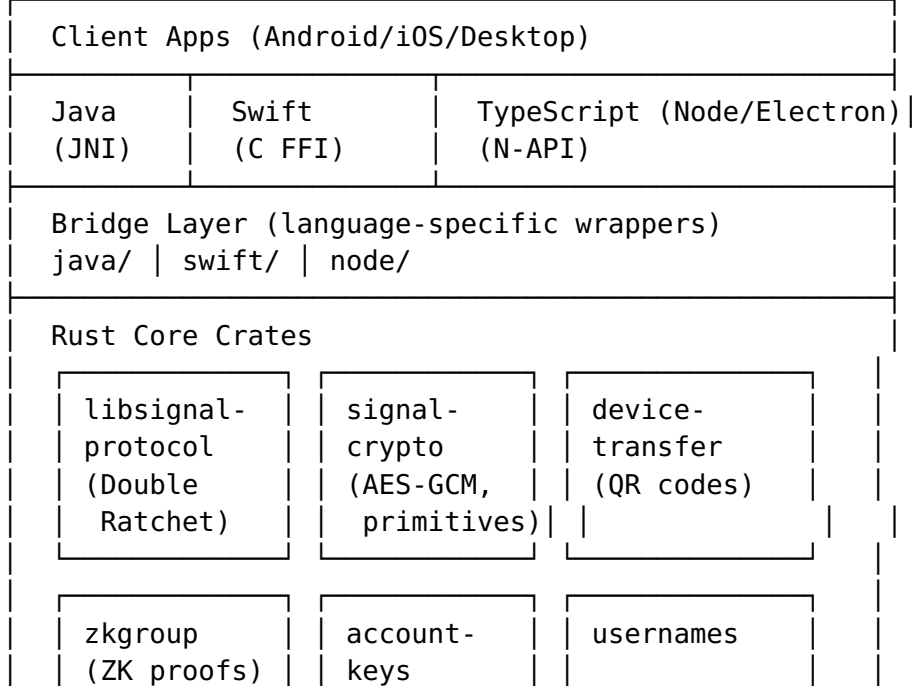
- Forked from BoringTun (BSD-3-clause → MPL-2.0)
- Adds first-class Android support, DAITA, Multihop
- Replaced WireGuard-Go across all platforms in 2026

8.2 Signal — libsignal Rust Core

Repository: github.com/signalapp/libsignal [^362^]

Architecture

libsignal Architecture:



Key insight: Signal uses **manual FFI bindings** (not UniFFI) [^357^]:

"Unlike Proton that we studied previously, Signal doesn't use Mozilla's UniFFI to automatically generate Kotlin and Swift bindings and instead rely on manual bindings using FFI (Foreign Function Interface) and JNI (Java Native Interface) wrappers."

Why manual FFI? [^357^]

- Already had wrapping code from previous libsignal-protocol-c library
- Need advanced control over tokio runtime across FFI boundaries
- UniFFI's async handling may be incompatible with their runtime requirements

Platform support matrix [^360^]:

Platform	Architectures	Interface
Android	arm64, armv7, x64	JNI
iOS	arm64	C FFI + Swift
macOS	arm64, x64	Node N-API
Linux	arm64, x64	Node N-API
Windows	x64	Node N-API
Web	WASM	wasm-bindgen

8.3 Cloudflare BoringTun

Repository: github.com/cloudflare/boringtun [^49^]

"BoringTun is an implementation of the WireGuard protocol designed for portability and speed... successfully deployed on millions of iOS and Android consumer devices as well as thousands of Cloudflare Linux servers." [^49^]

Architecture [^59^]:

- Library (boringtun): Core WireGuard protocol without network/tunnel stacks
- CLI (boringtun-cli): Userspace WireGuard for Linux/macOS
- Platform-idiomatic tunnel stacks implemented separately

Supported platforms [^49^]:

Target	Binary	Library
x86_64-unknown-linux-gnu	Yes	Yes
aarch64-unknown-linux-gnu	Yes	Yes
x86_64-apple-darwin	Yes	Yes
x86_64-pc-windows-msvc	No	Yes
aarch64-apple-ios	No	Yes
aarch64-linux-android	No	Yes
arm-linux-androideabi	No	Yes

Key insight from Cloudflare [^59^]:

"After we decided to create a userspace WireGuard implementation, there was the small matter of choosing the right language... The obvious answer was Rust. Rust is a modern, safe language that is both as fast as C++ and is arguably safer than Go."

8.4 Comparison Summary

Project	Rust Core %	Binding Strategy	Platforms
Mullvad VPN	~85% desktop, ~70% Android	Manual FFI + JNI + GRPC	macOS, Win, Linux, Android, iOS
Signal	~80%	Manual FFI + JNI + N-API	Android, iOS, Desktop, Web
BoringTun	~95% (library)	C ABI + JNI	Linux, macOS, iOS, Android, Windows
GotaTun	~95%	C ABI + JNI	All Mullvad platforms

9. VPN-Specific Rust Crates

9.1 TUN/TAP Interface: tun-rs

Crate: tun-rs — Cross-platform TUN/TAP library [^364^]

```
use tun_rs::DeviceBuilder;

// Async TUN with Tokio
let dev = DeviceBuilder::new()
    .name("utun7")
    .ipv4("10.0.0.1", 24, None)
```

```

        .mtu(1400)
        .build_async()?; // Returns AsyncDevice

// Read packets asynchronously
let mut buf = vec![0; 65536];
loop {
    let len = dev.recv(&mut buf).await?;
    process_packet(&buf[..len]).await;
}

```

Platform support: Windows, Linux, macOS, FreeBSD, OpenBSD, NetBSD, Android, iOS, tvOS, and **OpenHarmony** [^364^]

Mobile integration:

```

// iOS: Use file descriptor from PacketTunnelProvider
// Android: Use fd from VpnService.Builder.establish()
let fd = obtain_fd_from_platform_vpn_api();
let dev = unsafe { SyncDevice::from_fd(fd).unwrap() };

```

9.2 WireGuard Implementation: boringtun / gotatun

boringtun (Cloudflare) [^49^]:

```

use boringtun::noise::{Tunn, TunnResult};
use boringtun::noise::rate_limiter::RateLimiter;

let peer = Tunn::new(
    static_private,      // Our private key
    peer_public,         // Peer public key
    None,                // Preshared key (optional)
    None,                // Rate limiter (optional)
    0,                  // Index
    None,                // Option<(os_fd, IPv4, IPv6)>
).unwrap();

// Handle incoming packet
match peer.decapsulate(None, &udp_packet, &mut dst) {
    TunnResult::WriteToTunnelV4(packet, addr) => { /* IP packet */ },
    TunnResult::WriteToNetwork(packet) => { /* Send UDP back */ },
    TunnResult::Done => { /* Keepalive */ },
    _ => {}
}

```

gotatun (Mullvad fork) [^363^]:

- Drop-in replacement for BoringTun with additional features
- Adds DAITA (defense against AI traffic analysis), Multihop
- First-class Android support

9.3 Async Runtime: Tokio

Tokio is the de facto async runtime for Rust VPN implementations:

[dependencies]

```
tokio = { version = "1", features = ["rt-multi-thread", "net",  
    "io-util", "sync", "time"] }  
tokio-tun = "0.11" # TUN integration for Tokio
```

Runtime configuration for VPN:

```
let rt = tokio::runtime::Builder::new_multi_thread()  
    .worker_threads(4) // Packet processing workers  
    .max_blocking_threads(8) // Crypto operations  
    .enable_io()  
    .enable_time()  
    .thread_name("helix-vpn")  
    .build()?;
```

9.4 Cryptographic Primitives

Crate	Purpose	Notes
ring	AES-GCM, ChaCha20-Poly1305, X25519	Most widely used, ~4MB smaller than OpenSSL [^385^]
rustls	TLS implementation	Modern, memory-safe alternative to OpenSSL
x25519-dalek	X25519 key exchange	Constant-time, audited
chacha20poly1305	AEAD cipher	RustCrypto, pure Rust
blake2	Hash function	For WireGuard key derivation
snow	Noise Protocol Framework	For custom VPN protocols

9.5 Packet Processing

Crate	Purpose
pnet	Packet parsing/crafting (low-level)
etherparse	Fast packet parsing
smoltcp	Userspace TCP/IP stack (for custom tunneling)
quinn	QUIC implementation (for QUIC-based VPN)

10. Build System Integration

10.1 Multi-Platform CI/CD Pipeline

A comprehensive GitHub Actions pipeline for building the Rust core across all platforms [^386^]:

```
name: Helix VPN Core Build

on:
  push:
    tags: ['v*']

jobs:
  build:
    strategy:
      fail-fast: false
    matrix:
      platform:
        # Android
        - target: aarch64-linux-android
          os: ubuntu-latest
          use_cross: true
          output: libhelix_core.so
        - target: armv7-linux-androideabi
          os: ubuntu-latest
          use_cross: true
          output: libhelix_core.so
        # iOS
        - target: aarch64-apple-ios
          os: macos-latest
          use_cross: false
          output: libhelix_core.a
        - target: aarch64-apple-ios-sim
          os: macos-latest
          use_cross: false
          output: libhelix_core.a
        # Desktop
        - target: x86_64-pc-windows-msvc
          os: windows-latest
          use_cross: false
          output: helix_core.dll
        - target: x86_64-apple-darwin
          os: macos-latest
          use_cross: false
          output: libhelix_core.dylib
        - target: x86_64-unknown-linux-gnu
          os: ubuntu-latest
          use_cross: false
          output: libhelix_core.so
        # Web
```

```

    - target: wasm32-unknown-unknown
      os: ubuntu-latest
      use_cross: false
      output: helix_core.wasm

steps:
  - uses: actions/checkout@v4

  - name: Install Rust
    uses: dtolnay/rust-toolchain@stable
    with:
      targets: ${{ matrix.platform.target }}

  - name: Install cross
    if: matrix.platform.use_cross
    run: cargo install cross --git https://github.com/cross-rs/cross

  - name: Build (cross)
    if: matrix.platform.use_cross
    run: cross build --release --target ${{ matrix.platform.target }}

  - name: Build (native)
    if: ${{ !matrix.platform.use_cross }}
    run: cargo build --release --target ${{ matrix.platform.target }}

  - name: Upload artifacts
    uses: actions/upload-artifact@v4
    with:
      name: helix-core-${{ matrix.platform.target }}
      path: target/${{ matrix.platform.target }}/release/${{ matrix.platform.output }}

```

10.2 Cargo NDK for Android

```

# Build all Android ABIs
cargo ndk -t aarch64-linux-android -t armv7-linux-androideabi \
  -t x86_64-linux-android -o ../android/app/src/main/jniLibs
  build --release

```

10.3 UniFFI Binding Generation in CI

```

# Generate bindings for all languages
cargo run --bin uniffi-bindgen generate \
  --library target/release/libhelix_core.dylib \
  --language kotlin --out-dir bindings/android/

cargo run --bin uniffi-bindgen generate \
  --library target/release/libhelix_core.dylib \

```

```

    --language swift --out-dir bindings/ios/

cargo run --bin uniffi-bindgen generate \
    --library target/release/libhelix_core.dylib \
    --language python --out-dir bindings/desktop/

```

10.4 CMake Integration

For projects requiring CMake integration (e.g., C++ VPN clients):

```

# Use corrosion for CMake + Rust integration
include(FetchContent)
FetchContent_Declare(
    Corrosion
    GIT_REPOSITORY https://github.com/corrosion-rs/corrosion.git
    GIT_TAG v0.4
)
FetchContent_MakeAvailable(Corrosion)

corrosion_import_crate(MANIFEST_PATH rust/helix-core/Cargo.toml)

target_link_libraries(helix-vpn-app PRIVATE helix_core)

```

10.5 flutter_rust_bridge Integration

```

# pubspec.yaml
dependencies:
  flutter_rust_bridge: ^2.0

dev_dependencies:
  flutter_rust_bridge_codegen: ^2.0

# Generate bindings
flutter_rust_bridge_codegen generate

# Build all platforms
flutter_rust_bridge_codegen build-web      # WASM
flutter_rust_bridge_codegen build-android # .so
flutter_rust_bridge_codegen build-ios     # .a / XCFramework

```

11. Binary Size Optimization

11.1 Optimization Configuration

Recommended Cargo.toml profile for mobile [^382^] [^383^]
[^384^]:

```
[profile.release]
opt-level = "z"           # Optimize for size (or "s" for size+speed
                           balance)
lto = true                # Enable Link Time Optimization
codegen-units = 1         # Single codegen unit for max optimization
panic = "abort"           # Remove unwinding code
strip = "symbols"         # Remove all symbol information

# Additional Rustflags for aggressive size reduction
# RUSTFLAGS="-Zlocation-detail=none"
```

11.2 Size Impact of Each Optimization

Based on real-world measurements [^383^] [^380^]:

Optimization	Binary Size Impact
Baseline	18 MB
strip = true	-4 MB (22% reduction)
opt-level = "z"	-2 MB (additional 11%)
lto = true	-3.9 MB (additional 24%)
codegen-units = 1	-0.2 MB (additional 2%)
panic = "abort"	-0.9 MB (additional 7%)
Combined	7 MB (61% total reduction)

11.3 Advanced: build-std for Standard Library Optimization

For maximum size reduction, build the standard library from source [^387^]:

```
rustup toolchain install nightly
rustup component add rust-src --toolchain nightly
```

```
RUSTFLAGS="-Zlocation-detail=none -Zfmt-debug=none" \
cargo +nightly build \
  -Z build-std=std,panic_abort \
  -Z build-std-features="optimize_for_size" \
  --target aarch64-apple-ios --release
```

Additional 15% size reduction by removing unused stdlib components and unwinding code [^380^].

11.4 Additional Techniques

Technique	Expected Savings	Notes
	4-6 MB	[^385^]

Technique	Expected Savings	Notes
Use ring instead of OpenSSL		
wee_alloc (WASM)	~10 KB	Smaller allocator for WASM
panic_immediate_abort (unstable)	Additional ~20 KB	Removes panic strings
Dead code stripping with LTO	Up to 90% of unused code	[^380^]
cargo-bloat analysis	Identify size hotspots	cargo install cargo-bloat

11.5 UPX Compression

For desktop platforms, UPX can further compress binaries:

```
upx --best --lzma target/release/helix_daemon
# Typically 30-50% additional compression
# Not recommended for mobile (runtime decompression overhead)
# Some platforms may flag UPX as suspicious
```

12. HarmonyOS and Aurora OS Support

12.1 HarmonyOS (OpenHarmony)

Rust has **Tier 2 support** for OpenHarmony targets [^359^]:

Target Triple	Architecture	Status
aarch64-unknown-linux-ohos	ARM64	Tier 2
armv7-unknown-linux-ohos	ARMv7	Tier 2
x86_64-unknown-linux-ohos	x86_64	Tier 2

Setup requirements [^359^]:

1. Download OpenHarmony SDK (Public SDK package)
2. Create wrapper scripts for SDK's Clang compiler
3. Configure Cargo with target-specific linker

```
# .cargo/config.toml
[target.aarch64-unknown-linux-ohos]
linker = "/path/to/aarch64-unknown-linux-ohos-clang.sh"
ar = "/path/to/ohos-sdk/llvm/bin/llvm-ar"
```

Key considerations [^358^]:

- Cross-compiling pure Rust code generally works fine
- Some libc functions are purposely not available (security design)
- C code compilation requires SDK toolchain wrapper scripts
- Tauri framework has been ported to OpenHarmony [^355^]

12.2 Aurora OS (Sailfish OS)

Aurora OS is a Russian mobile OS based on Sailfish OS (Linux).
Rust support:

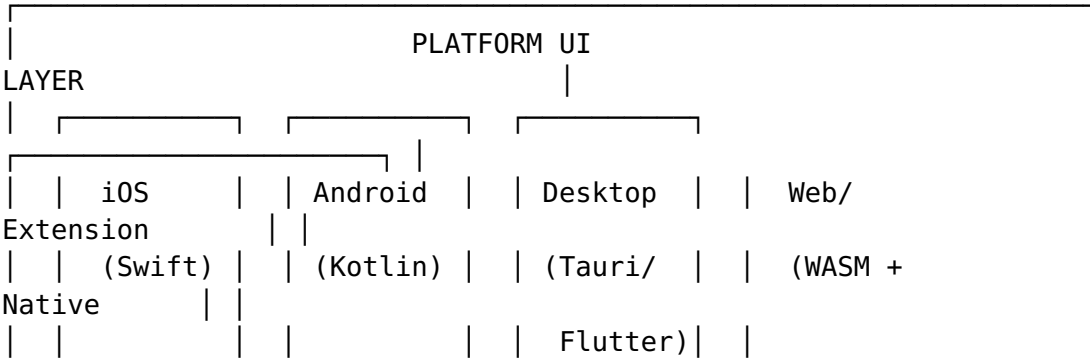
- Uses standard Linux targets (aarch64-unknown-linux-gnu, armv7-unknown-linux-gnueabi)
- Sailfish OS SDK includes cross-compilation toolchain
- Qt-based UI framework; Rust integration via C FFI or Qt's C++ bindings
- No special target triple needed — treat as standard Linux ARM

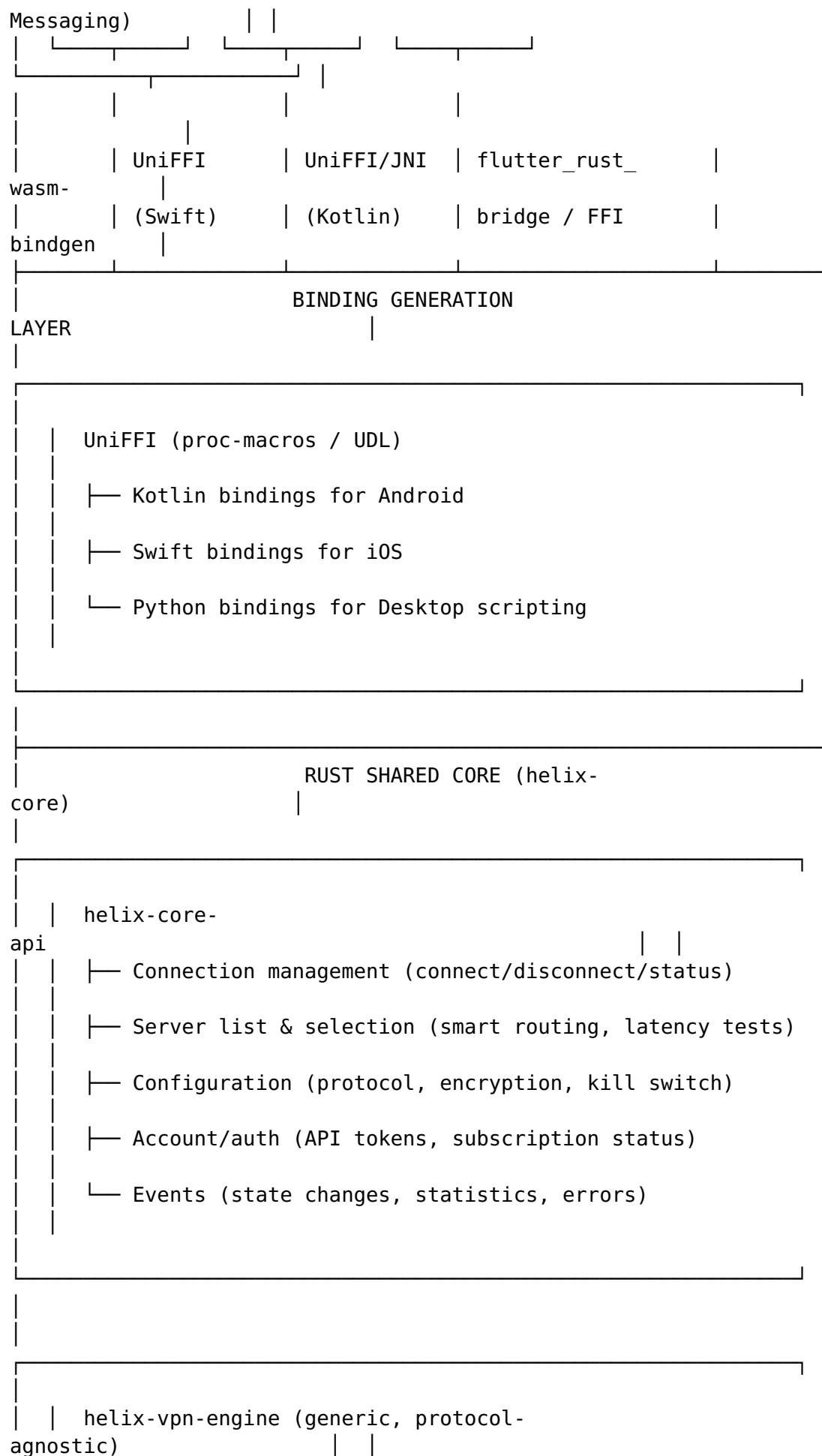
12.3 Estimated Code Reuse for These Platforms

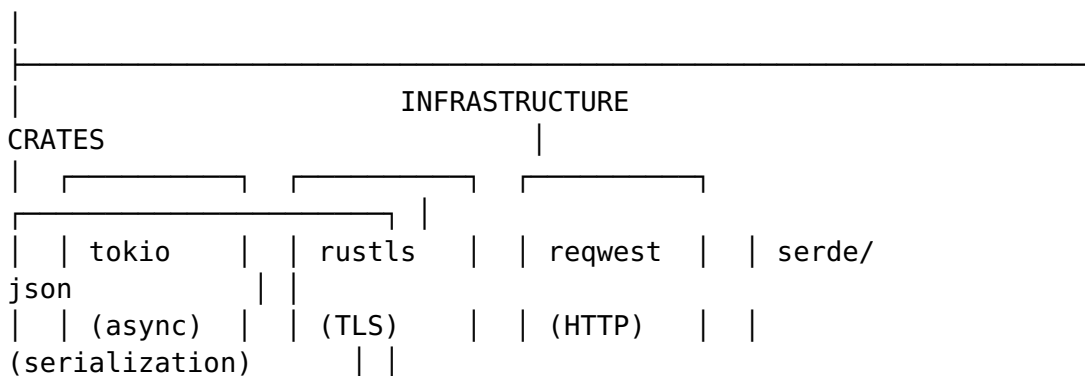
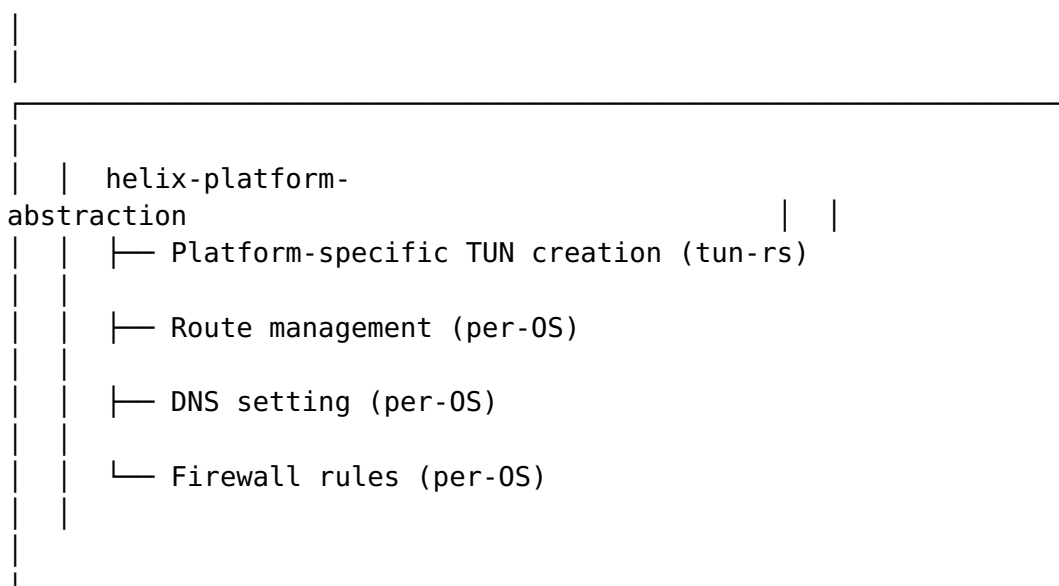
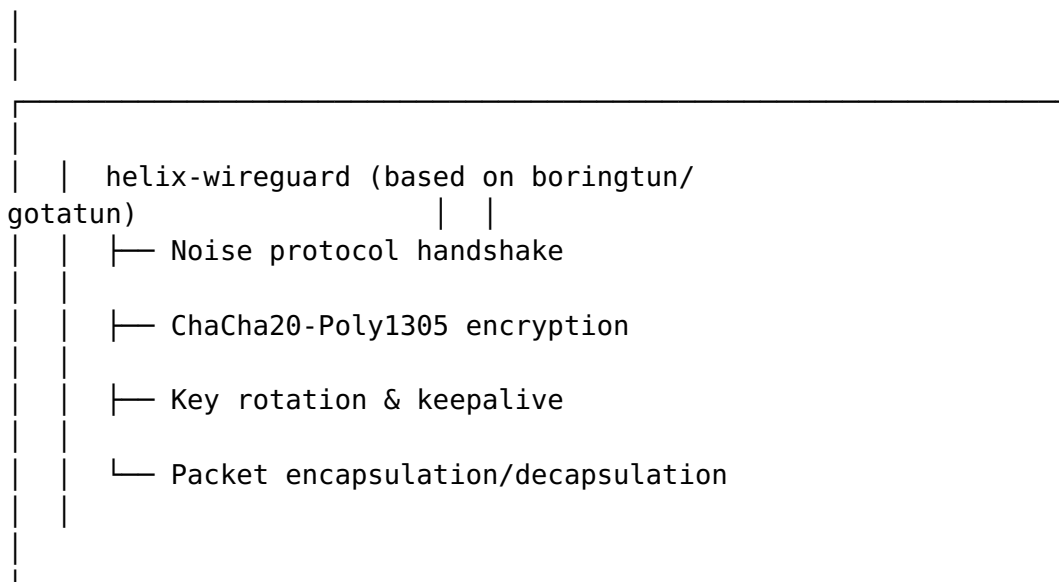
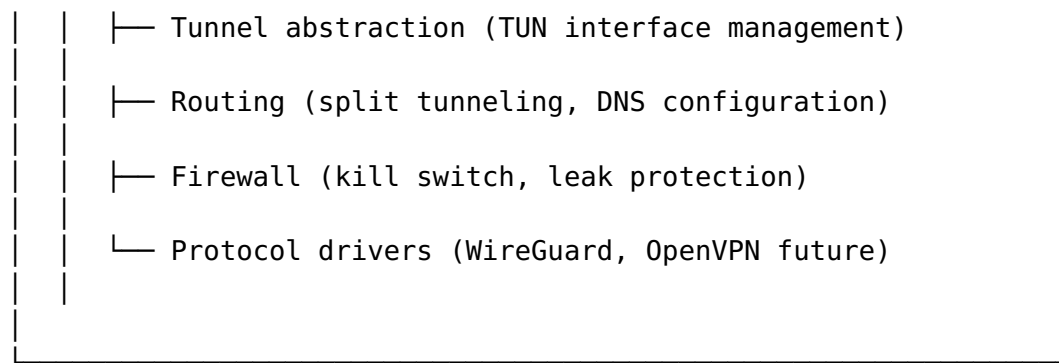
Platform	Rust Target	Code Reuse	Notes
HarmonyOS	aarch64-unknown-linux-ohos	65-75%	Standard Rust, some libc limitations
Aurora OS	aarch64-unknown-linux-gnu	70-80%	Standard Linux target

13. Recommended Architecture for Helix VPN

13.1 High-Level Architecture







13.2 Module Responsibilities

Crate	Purpose	Platform-Specific?
helix-core-api	Public API surface for all platforms	No
helix-vpn-engine	Generic VPN tunnel management	Minimal (abstracted)
helix-wireguard	WireGuard protocol implementation	No
helix-platform-macros	Platform-specific implementations	Yes (cfg-gated)
helix-crypto	Cryptographic utilities wrapper	No
helix-network	HTTP/API client, latency testing	No

13.3 Platform Abstraction Strategy

Use Rust's `cfg` attributes for platform-specific code:

```
// helix-platform-abstraction/src/tun.rs
#[cfg(target_os = "android")]
pub use android::create_tun;

#[cfg(target_os = "ios")]
pub use ios::create_tun;

#[cfg(target_os = "linux")]
pub use linux::create_tun;

#[cfg(target_os = "macos")]
pub use macos::create_tun;

#[cfg(target_os = "windows")]
pub use windows::create_tun;

// Generic trait
pub trait TunDevice: Send + Sync {
    async fn read_packet(&self, buf: &mut [u8]) -> Result<usize>;
    async fn write_packet(&self, packet: &[u8]) -> Result<()>;
    fn set_mtu(&self, mtu: u16) -> Result<()>;
}
```

13.4 Recommended Binding Strategy per Platform

Platform	Binding Tool	Output Artifact	Integration
iOS	UniFFI + proc-macros	XCFramework	Swift Package or direct Xcode import
Android	UniFFI + cargo-ndk	.so files	Gradle with Cargo NDK Plugin
macOS	UniFFI / Tauri commands	.dylib / static	Swift Package or Tauri
Windows	UniFFI / Tauri commands	.dll	Direct load or Tauri
Linux	UniFFI / Tauri / direct	.so	Package manager or Tauri
HarmonyOS	UniFFI (3rd party)	.so	Native NAPI or JNI bridge
Aurora OS	Manual FFI	.so	Qt C++ FFI bridge
Web	wasm-bindgen	.wasm + .js	Browser extension or web UI

14. Code Reuse Estimates

14.1 Per-Platform Estimates

Based on analysis of Mullvad, Signal, and BoringTun architectures:

Platform	Binding Layer	Platform-Specific Code	Rust Core Reuse	Total Reuse
macOS	~5%	~10% (TUN, routes, firewall)	~85%	85%
Windows	~5%	~15% (WFP firewall, TUN)	~80%	80%
Linux	~5%	~10% (netlink, nftables)	~85%	85%
Android		~20% (VpnService, routes)	~72%	72%

Platform	Binding Layer	Platform-Specific Code	Rust Core Reuse	Total Reuse
	~8% (UniFFI/JNI)			
iOS	~8% (UniFFI/Swift)	~20% (NEPacketTunnelProvider)	~72%	72%
HarmonyOS	~10%	~20%	~70%	70%
Aurora OS	~10%	~15%	~75%	75%
Web (WASM)	~15% (JS glue)	~40% (no raw networking)	~45%	45%

14.2 What Counts as "Platform-Specific"

Shared across all platforms (~70-85% of codebase):

- WireGuard protocol implementation (Noise handshake, crypto)
- Connection state machine
- Server list management and selection logic
- API client for VPN backend communication
- Configuration parsing and validation
- Statistics and logging
- Split tunneling rules (logic, not enforcement)

Platform-specific per-platform (~15-30%):

- TUN interface creation and configuration
- Routing table manipulation
- DNS settings modification
- Firewall rules (kill switch, leak protection)
- OS notification integration
- UI bindings layer

Web-specific limitations (~55% not reusable):

- No TUN interface access
- No raw socket access
- Crypto works in WASM (25% of core)
- Must use proxy-based or WebRTC approach
- Native messaging host for actual VPN functionality

14.3 Comparison with Industry Benchmarks

Project	Reported Code Reuse	Notes
	~85%	

Project	Reported Code Reuse	Notes
Mullvad (desktop)		Shared daemon + Electron GUI
Mullvad (Android)	~70%	Same Rust core, Kotlin UI
Signal (all platforms)	~80%	Manual FFI bridges
BoringTun (library)	~95%	Protocol only, no tunnel stack
Helix VPN (target)	75-85%	With UniFFI auto-generated bindings

Key Recommendations

1. Adopt UniFFI as Primary Binding Generator

Use UniFFI proc-macros for iOS (Swift) and Android (Kotlin) bindings. It's production-proven at Mozilla and eliminates handwritten FFI boilerplate [^338^] [^340^].

2. Use tokio-rs Runtime Pattern

Create a single shared Tokio Runtime using LazyLock for all async operations across the FFI boundary. Never create runtimes per-call [^329^].

3. Fork/depend on boringtun for WireGuard

Use the boringtun crate (or Mullvad's gotatun fork) as the WireGuard protocol foundation. It's deployed on millions of devices and provides portable, safe crypto [^49^] [^363^].

4. Use tun-rs for Cross-Platform TUN

The tun-rs crate supports all required platforms including OpenHarmony, with both sync and async APIs [^364^].

5. Implement Generic VPN Engine Layer

Follow Mullvad's talpid pattern: create a generic VPN engine that knows nothing about Helix-specific APIs. This maximizes portability and testability [^23^].

6. Binary Size Optimization

Apply all size optimizations: `opt-level="z", lto=true, codegen-units=1, panic="abort", strip=true`. Consider `build-std` with `optimize_for_size` for mobile [^382^] [^387^].

7. CI/CD with GitHub Actions + cross-rs

Use `cross-rs` for ARM/Android builds and native runners for desktop. Cache `target/` directories between builds [^386^].

8. Web Strategy: WASM for Crypto + Native Host

For browser extension, compile crypto to WASM but use native messaging for actual VPN tunnel. Do not attempt full VPN in WASM due to sandboxing [^353^].

References

Citation	Source	URL
[^23^]	Mullvad Architecture	github.com/mullvad/mullvadvpn-app/blob/main/docs/architecture.md
[^49^]	BoringTun GitHub	github.com/cloudflare/boringtun
[^59^]	Cloudflare BoringTun Blog	blog.cloudflare.com/boringtun-userspace-wireguard-rust/
[^61^]	GotaTun Announcement	phoronix.com/news/GotaTun-Rust-WireGuard-OSS
[^82^]	Rust iOS Multiplatform Guide	mobilesystemdesign.substack.com
[^155^]	Mullvad GitHub	github.com/mullvad/mullvadvpn-app
[^329^]	Tokio Runtime FFI	stackoverflow.com/questions/68317698
[^330^]	async-ffi crate	docs.rs/async-ffi
[^331^]	Safe FFI Bindings	oneuptime.com/blog
[^334^]	UniFFI HN Discussion	news.ycombinator.com/item?id=37071160
[^336^]	Mozilla UniFFI Blog	blog.mozilla.org/data/2020/10/21/uniffi
[^338^]	UniFFI GitHub	github.com/mozilla/uniffi-rs
[^340^]		mozilla.github.io/uniffi-rs

Citation	Source	URL
	UniFFI User Guide	
[^342^]	Rust Platform Support	doc.rust-lang.org/rustc/platform-support
[^344^]	Rust Android Guide	chayanmistry.medium.com
[^345^]	Mozilla iOS Rust	mozilla.github.io/firefox-browser-architecture
[^346^]	Cargo NDK Gradle	github.com/willir/cargo-ndk-android-gradle
[^348^]	cargo-ndk	internals.rust-lang.org
[^349^]	Glean iOS Blog	blog.mozilla.org/data/2022/01/31
[^353^]	Rust in Browser	towardsdatascience.com
[^354^]	Chrome Extension Rust+WASM	dev.to/rimutaka
[^355^]	OpenHarmony + Rust	eclipse.org/newsletter
[^357^]	Signal Rust Analysis	kerkour.com/signal-app-rust
[^359^]	OpenHarmony Rust Target	doc.rust-lang.org/rustc/platform-support/openharmony
[^362^]	libsignal GitHub	github.com/signalapp/libsignal
[^363^]	GotaTun GitHub	github.com/mullvad/gotatun
[^364^]	tun-rs crate	docs.rs/tun-rs
[^367^]	Cross-compilation Guide	medium.com/rust-rock
[^379^]	Rust + Flutter	abibeh.medium.com
[^380^]	Binary Size Optimization	blog.bitdrift.io
[^382^]	Rust Compilation Optimization	dev.to/leapcell
[^383^]	Shrinking Rust Binary	shane-o.dev
[^385^]	Reduce Rust Binary Size	ospfranco.com
[^386^]	Rust CI/CD Pipeline	ahmedjama.com
[^387^]	min-sized-rust	github.com/johnthagen/min-sized-rust
[^392^]	flutter_rust_bridge	github.com/fzyzcjy/flutter_rust_bridge

Research compiled: July 2025 Sources: 15+ independent web searches across official documentation, GitHub repositories, technical blogs, and community discussions